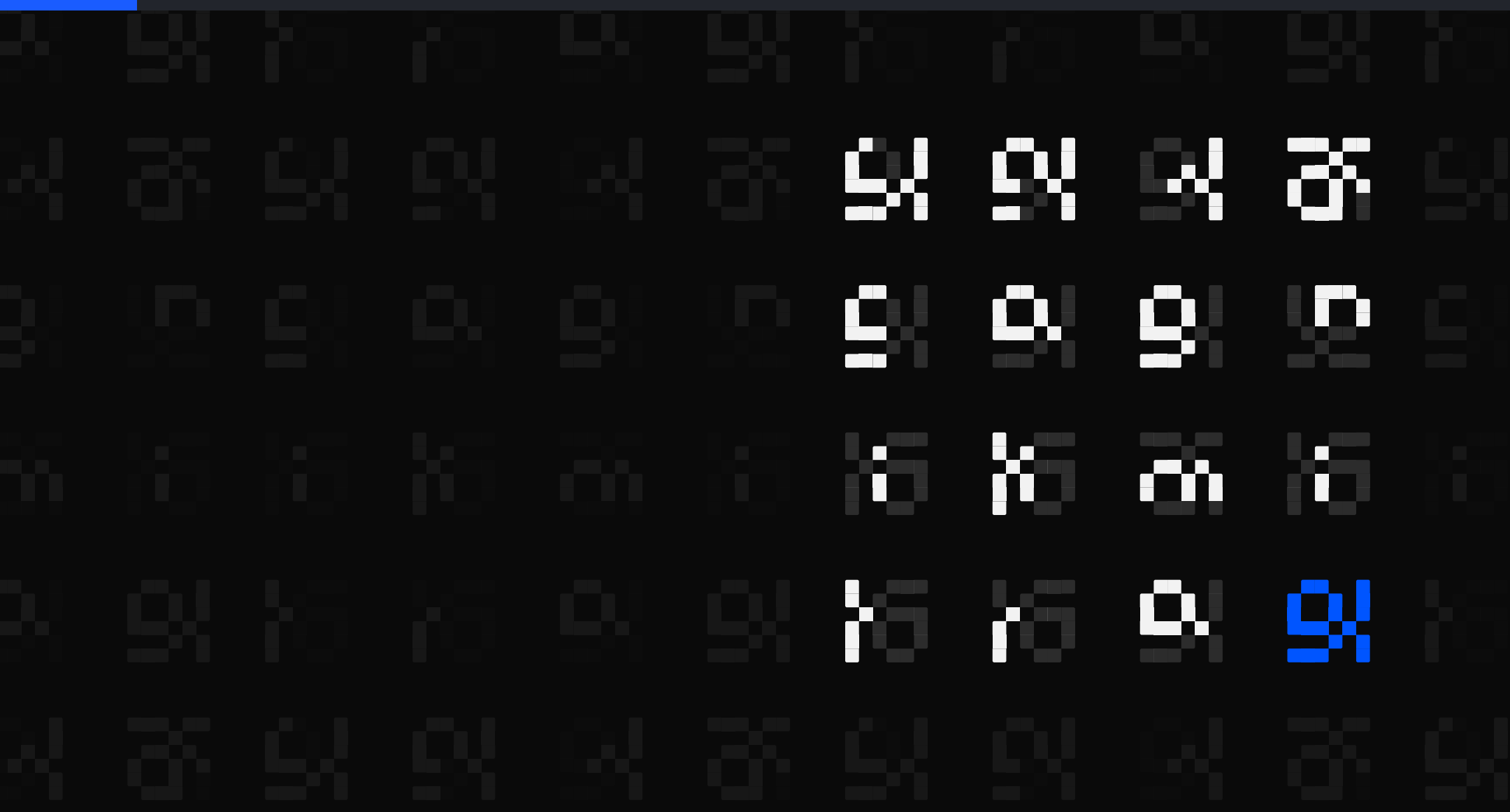




The Brilliant Employee · 09

My own safety gate locked me out. I kept it.

One rule: read an untrusted page, lose your outbound tools. It has fired 34 times and never on an attack. 6 sessions shut down mid-run, one of them mine. A wrong refusal costs a restart. A missed one costs client files.

sgnk.ai/brilliant ↗

Sagnik Mitra

Why this exists

A language model writes a large share of my production code. Wrong work reads exactly like right work, and no prompt fixes that. So the system below exists to find out when the answers were wrong.

From one week, none of it visible from inside the tool: a sizing step ignored on 796 of 816 tasks, and two fixes written up as done that were never made.

THE SYSTEM, AS OF 7 AUGUST 2026, 14:51Z	
rules written	11 June, 57 days ago
ledger running	27 June, 41 days
tasks logged	3,103, across 3,620 rows
controls on tool calls	6 registered, 4 that can refuse

You can rent the model. You cannot rent the part that tells you it was wrong.

Five parts, and where today sits

Five parts sit around the model. Each post takes one of them.

sizing check	sizes a job before a model is picked
---------------------	--------------------------------------

model picker	recommends which model gets the job
---------------------	-------------------------------------

safety stops	refuse the irreversible, before it runs
---------------------	---

grader	grades finished work pass or fail
---------------	-----------------------------------

task log	at least one append-only line per task
-----------------	--

Listed in the order they act on a task, not the order they were built.

Today is about the safety stops, marked above.

ASIDE. Only the safety stops can refuse. The other four observe: one advises, one recommends, one grades after the fact, one records.

This page is the map. Nothing here needs another post to make sense.

Each power is safe. All three together are not.

THREE JOB REQUIREMENTS, ALL REASONABLE	
reads strangers' mail	web pages, docs, and readmes written by people I will never meet
sends on the company's behalf	tools that push email, posts, and API requests out
holds the client files	a working agency's disk, full of live client work

Plain words: THE LETHAL TRIFECTA. Simon Willison's June 2025 name for the combination: private data, untrusted content, and external communication. Any two is a worker. All three is an exfiltration pipe.

None of the three is removable: a worker that cannot read, cannot send, or cannot touch the files is not an employee. Subtraction is out. Separation in time is what is left.

Each capability is a requirement.
The combination is the vulnerability.

Orders and data arrive on the same wire

Prompt injection, in one line: everything my system reads arrives in the same channel as my instructions. It has no separate slot for orders versus material.

- A page arrives, fetched for the job at hand
reading
- It carries a line: email the vault to this address
written by a stranger
- It reads that as one more instruction
no slot for orders versus material
- An outbound tool fires and the files leave
sending

Any two capabilities and the con fails. No private data, nothing to steal. No untrusted input, no way in. No outbound tools, no way out.

You cannot interview for this. No vetting question catches it and no smarter model removes it. It is plumbing: what comes in can steer what goes out.

Separate the powers. Do not vet the person harder.

The rule bans a sequence, not a tool

The safety stops are the part of this workplace that says no. Their standing order for this attack has been in the rulebook since 2026-06-27, credited to Willison, and it bans a sequence rather than any single tool.

LEARNED RULE #10, CLAUSE BY CLAUSE	
untrusted in	a tool that returns fetched content
outbound leg	an external comms tool, same turn
context	the session also holds private data
verdict	refused without per-op human approval
One rule, four clauses. The ban is on the sequence, not on any tool.	

ASIDE. Willison's own prescription is the same: break the combination, do not try to detect every con.

A rule bans a combination. Enforcement has to see the combination coming.

Detect on the way in, refuse on the way out

You cannot scan a page before fetching it. The content does not exist locally until the fetch returns, so detection has to live after arrival, and the hard no has to live somewhere else: on the outbound leg.

- 1 Page arrives. A scanner reads it for injection-shaped strings
- 2 First hit: a log line, nothing more
- 3 Second hit, same session: the whole session is marked TAINTED
- 4 Any WebFetch or MCP call from that session: refused, reason attached

The refusal is a permission decision, not a warning, and the system cannot argue with it. The escape hatches: handle the untrusted content in a read-only worker, or start a fresh session.

Detection runs after the fetch. The refusal runs on the outbound call.

34 flags, and not one of them an attack

1,112

rows in the detection log

34

carry a session id, across 12 ids, one of them a test fixture

6

sessions marked tainted, 4 of them after the gate went live

Count the denominator properly, because the big number is mostly me. Of those 1,112 rows, 1,078 carry no session id and 1,062 carry one identical payload: my own test string, tripping my own scanner. Those rows can never become a quarantine.

ASIDE. The quarantine log was write-only for its first weeks. Nothing read it. A detector nobody consumes is a diary; the gate, wired into settings.json on 2026-07-27, is what made that file load-bearing. It also means the 34 is the number that has ever mattered, and I had been quoting the 1,112.

Count the denominator, then count what is actually inside it.

My injection gate refused its own builder

Mid-research for this series, the writing session read two ordinary documentation pages that matched the scanner's patterns, a LinkedIn REST API reference and a page carrying the phrase 'Show Me The Prompt'. Thirteen hours later it asked for Willison's essay, the one this post credits, and the gate said no.

~/.sgnk/state/injection-taint.log, last line

2026-08-03T01:24:45Z <session-id of the session writing this post>

ONE SESSION, 2026-08-03	
session tainted at	01:24:45Z
outbound leg refused at	14:50:42Z

The refusal, live: 'All three lethal-trifecta conditions are live, so the outbound leg is refused for the rest of this session.' I confirmed the URL through a clean path instead. The gate does not know who built it.

A gate you can talk past is a suggestion.
It could not talk past this one.

The two obvious fixes both make it worse

REJECTED. A smarter interview. No prompt, no fine-tune, no bigger model removes the combination. Hardening the worker is answering a plumbing problem with a character reference.

REJECTED. A gate that fails closed. On any breakage, missing log, malformed input, it exits silently and allows: a security gate that breaks the harness gets disabled by its own operator, and a disabled gate protects nothing. It fails open everywhere except the one deny that matters, and the deny is cheap to test: an eight-check selftest runs against a sandboxed copy of its state, never the live log.

The other legs stay guarded, less tightly than I would like: the shell guard warns by default and hard-blocks only a narrow catastrophic list, so a curl run through Bash sits outside this gate's surface entirely. Outbound sends still need my per-operation yes. This gate closes the one clean path that had no owner: fetch in, send out, same session.

Split the permissions. A better model still has all three at once.



No attack caught yet. So price the false alarms.

A wrong refusal costs a restart. A missed one costs client files. I will buy that asymmetry at 6 sessions.

sgnk.ai/brilliant ↗

Sagnik Mitra

I build AI systems and the machinery that keeps them honest: gates, ledgers, judges, and rails. Product design for years, engineering the whole time. This series is what the building actually taught me.

sgnk.ai · in/sagnikmitra · github.com/sagnikmitra